

```
1  -- =====
2  -- SECTION A : Basic Queries
3  -- Database   : Celebal_Week2
4  -- Table      : dbo.Superstore
5  -- Tool       : SQL Server Management Studio (SSMS)
6  -- Author     : ARATI THORAT | Celebal Internship - Week 2
7  -- =====
8
9  USE Celebal_Week2;
10 GO
11
12 -- -----
13 -- A1. View ALL columns from the Superstore table
14 -- PURPOSE : Get a full picture of the raw dataset
15 -- -----
16 SELECT *
17 FROM   dbo.Superstore;
18
19 /*
20  OBSERVATION:
21  The table contains 9,994 rows and 21 columns covering
22  order details, customer info, product info, and financials.
23  All data is in a single flat/denormalised table.
24 */
25
26
27 -- -----
28 -- A2. View only important columns (selective projection)
29 -- PURPOSE : Reduce noise, focus on key fields
30 -- -----
31 SELECT
32     Row_ID,
33     Order_ID,
34     Order_Date,
35     Ship_Date,
36     Ship_Mode,
37     Customer_Name,
38     Segment,
39     Region,
40     Category,
41     Sub_Category,
42     Product_Name,
43     Sales,
44     Quantity,
45     Discount,
46     Profit
47 FROM   dbo.Superstore;
48
49 /*
```

```
50  OBSERVATION:
51  Projecting only the business-relevant columns makes the
52  result easier to read and speeds up query execution.
53  */
54
55
56  -- -----
57  -- A3. Total row count
58  --     PURPOSE : Confirm data load was complete
59  -- -----
60  SELECT COUNT(*) AS Total_Records
61  FROM    dbo.Superstore;
62
63  /*
64  OBSERVATION:
65  Expected result → 9,994 rows.
66  If any number other than 9,994 appears, the CSV import
67  may have been incomplete and needs to be re-imported.
68  */
69
70
71  -- -----
72  -- A4. Inspect column names and data types from schema
73  --     PURPOSE : Understand the table structure before querying
74  -- -----
75  SELECT
76      COLUMN_NAME,
77      DATA_TYPE,
78      CHARACTER_MAXIMUM_LENGTH,
79      IS_NULLABLE
80  FROM    INFORMATION_SCHEMA.COLUMNS
81  WHERE   TABLE_NAME   = 'Superstore'
82  AND     TABLE_SCHEMA = 'dbo'
83  ORDER BY ORDINAL_POSITION;
84
85  /*
86  OBSERVATION:
87  Key columns and their expected types:
88      Order_Date → date / varchar (check if parsed correctly)
89      Sales      → float / decimal
90      Profit     → float / decimal
91      Quantity   → int
92      Discount   → float
93  If dates are stored as VARCHAR instead of DATE,
94  conversion functions (CONVERT / TRY_CONVERT) must be used.
95  */
96
97
98  -- -----
```

```
99 -- A5. DISTINCT values in categorical columns
100 --     PURPOSE : Understand the domain of key dimensions
101 -- -----
102
103 -- Unique customer segments
104 SELECT DISTINCT Segment FROM dbo.Superstore;
105
106 -- Unique regions
107 SELECT DISTINCT Region FROM dbo.Superstore;
108
109 -- Unique ship modes
110 SELECT DISTINCT Ship_Mode FROM dbo.Superstore;
111
112 -- Unique product categories
113 SELECT DISTINCT Category FROM dbo.Superstore;
114
115 -- Unique sub-categories
116 SELECT DISTINCT Sub_Category
117 FROM     dbo.Superstore
118 ORDER BY Sub_Category;
119
120 -- Unique countries
121 SELECT DISTINCT Country FROM dbo.Superstore;
122
123 /*
124     OBSERVATION:
125     Segments    : Consumer, Corporate, Home Office
126     Regions     : East, West, Central, South
127     Ship Modes  : Standard Class, Second Class, First Class, Same Day
128     Categories  : Furniture, Office Supplies, Technology
129     Sub-Cats    : 17 unique sub-categories
130     Country     : United States only (domestic dataset)
131 */
132
133
134 -- -----
135 -- A6. First 10 rows (quick data preview)
136 --     PURPOSE : Sanity-check the data after import
137 -- -----
138 SELECT TOP 10 *
139 FROM     dbo.Superstore;
140
141 /*
142     OBSERVATION:
143     The first 10 rows give a quick look at data formatting.
144     Check that dates, decimals, and text fields look correct.
145 */
146
147
```

```

148 -- 
149 -- A7. Date range of orders
150 --     PURPOSE : Understand the time span of the dataset
151 -- 
152 SELECT
153     MIN(Order_Date) AS Earliest_Order,
154     MAX(Order_Date) AS Latest_Order
155 FROM dbo.Superstore;
156
157 /*
158     OBSERVATION:
159     Dataset spans from 2014 to 2017 (4 years of sales data).
160     This enables multi-year trend analysis.
161 */
162
163
164 -- 
165 -- A8. Sales and Profit range
166 --     PURPOSE : Understand the financial spread of the data
167 -- 
168 SELECT
169     ROUND(MIN(Sales), 2) AS Min_Sale,
170     ROUND(MAX(Sales), 2) AS Max_Sale,
171     ROUND(AVG(Sales), 2) AS Avg_Sale,
172     ROUND(MIN(Profit), 2) AS Min_Profit,
173     ROUND(MAX(Profit), 2) AS Max_Profit,
174     ROUND(AVG(Profit), 2) AS Avg_Profit
175 FROM dbo.Superstore;
176
177 /*
178     OBSERVATION:
179     - Min Sale ≈ $0.44 | Max Sale ≈ $22,638
180     - Some Profit values are NEGATIVE → loss-making transactions exist.
181     - The wide range suggests outlier orders worth investigating.
182 */
183
184
185 -- 
186 -- A9. NULL / missing value check
187 --     PURPOSE : Identify data quality issues early
188 -- 
189 SELECT
190     SUM(CASE WHEN Row_ID IS NULL THEN 1 ELSE 0 END) AS Null_RowID,
191     SUM(CASE WHEN Order_ID IS NULL THEN 1 ELSE 0 END) AS Null_OrderID,
192     SUM(CASE WHEN Customer_Name IS NULL THEN 1 ELSE 0 END) AS Null_CustomerName,
193     SUM(CASE WHEN Sales IS NULL THEN 1 ELSE 0 END) AS Null_Sales,
194     SUM(CASE WHEN Profit IS NULL THEN 1 ELSE 0 END) AS Null_Profit,

```

```

195     SUM(CASE WHEN Quantity IS NULL THEN 1 ELSE 0 END) AS      ↗
        Null_Quantity,
196     SUM(CASE WHEN Discount IS NULL THEN 1 ELSE 0 END) AS      ↗
        Null_Discount,
197     SUM(CASE WHEN Region IS NULL THEN 1 ELSE 0 END) AS Null_Region,
198     SUM(CASE WHEN Category IS NULL THEN 1 ELSE 0 END) AS      ↗
        Null_Category
199 FROM dbo.Superstore;
200
201 /*
202     OBSERVATION:
203     All critical columns should return 0 nulls.
204     If any null count > 0, those records need cleaning
205     before running aggregation queries.
206 */
207
208
209 -- -----
210 -- A10. Duplicate Row_ID check
211 --     PURPOSE : Verify primary key uniqueness after import
212 -- -----
213 SELECT
214     Row_ID,
215     COUNT(*) AS Occurrence_Count
216 FROM   dbo.Superstore
217 GROUP BY Row_ID
218 HAVING COUNT(*) > 1;
219
220 /*
221     OBSERVATION:
222     An empty result = no duplicates = clean import.
223     If duplicates exist, the CSV import included the header
224     row twice or had repeated rows – needs cleaning.
225 */
226
227
228 -- -----
229 -- A11. Count of unique customers, products, and orders
230 --     PURPOSE : Understand dataset cardinality
231 -- -----
232 SELECT
233     COUNT(DISTINCT Customer_ID) AS Unique_Customers,
234     COUNT(DISTINCT Product_ID) AS Unique_Products,
235     COUNT(DISTINCT Order_ID) AS Unique_Orders,
236     COUNT(DISTINCT State) AS Unique_States,
237     COUNT(DISTINCT City) AS Unique_Cities
238 FROM   dbo.Superstore;
239
240 /*

```

```
241     OBSERVATION:
242     ~793 unique customers
243     ~1,862 unique products
244     ~5,009 unique orders
245     49 states covered
246     531 cities
247 */
```